

Assignment 4

Introduction to Blockchain (ITB)

Semester: Spring 2026

Instructor: Ahmed Akhtar

Date: Tuesday, 28th April 2026

Members

1. Zuha Aqib, 26106
2. Izza Sohail Khan, 28996

Table of Contents

Members.....	1
Table of Contents.....	1
Question 1.....	3
Part (a).....	3
Part (b).....	3
Part (c).....	3
Part (d).....	4
Question 2.....	5
Part (a).....	5
Part (b).....	5
Part (c).....	5
Part (d).....	6
Part (e).....	6
Question 3.....	7
Part (a).....	7
Part (b).....	7
Part (c).....	8
Part (d).....	9
Question 4.....	10
Part (a).....	10
Part (b).....	10
Part (c).....	11
Part (d).....	11
Part (e).....	12
Question 5.....	13
Part (a).....	13

Part (b).....	13
Part (c).....	13
Part (d).....	14
Part (e).....	14
Question 6.....	16
Part (a).....	16
Part (b).....	16
Part (c).....	16
Part (d).....	17
Part (e).....	17

Question 1

SegWit Fundamentals: Transaction Structure, Malleability, and Witness Programs (10)
Segregated Witness (SegWit) changed the way digital signatures and script arguments are stored inside Bitcoin transactions.

Part (a)

a) Explain what data SegWit removes from the traditional scriptSig and where this data goes in a SegWit transaction. Why does this separation reduce transaction malleability? (2)

Segregated Witness (SegWit) removes the digital signature data and script arguments from the transaction's identifier calculation. In a legacy transaction, these elements were located within the scriptSig field. In a SegWit transaction, this data is moved to a separate structure called the witness field, which sits outside the part of the transaction used to calculate the TXID. This separation reduces transaction malleability because the TXID is no longer dependent on the witness data. Since digital signatures can be mathematically altered without changing their validity, moving them out of the TXID calculation ensures that a transaction's ID remains constant even if the signature is tweaked by an intermediary.

Part (b)

b) Consider a SegWit pay-to-witness-public-key-hash (P2WPKH) output with witness program:
 $0x00 \parallel \text{HASH160}(\text{pubkey})$.

Construct the spending transaction fields:

- The scriptSig of the input,
- The witness stack items,
- A short explanation of how the P2WPKH script is evaluated. (4)

For a Pay-to-Witness-Public-Key-Hash spending transaction, the scriptSig of the input is left completely empty. The witness stack contains two distinct items, which are the cryptographic signature followed by the full public key corresponding to the hash in the witness program. When evaluating the script, the Bitcoin node recognizes the version 0 witness program and identifies it as P2WPKH. The node then takes the public key from the witness stack, hashes it, and compares it to the hash provided in the output script. The node verifies the public key hash and then checks the signature using the public key. Since P2WPKH is SegWit v0, the signature scheme here is the normal Bitcoin ECDSA signature, not Taproot/Schnorr.

Part (c)

c) SegWit introduces the concept of “witness weight” and defines the block weight metric:
 $\text{weight} = (\text{non-witness bytes} \times 4) + (\text{witness bytes} \times 1)$.

Explain why witness data is discounted and give one practical effect on transaction fee incentives. (2)

Witness data is discounted using the block weight metric because it does not contribute to the permanent growth of the Unspent Transaction Output (UTXO) set in the same way that non-witness data does. By calculating weight as $(\text{non-witness} \times 4) + (\text{witness} \times 1)$, the protocol effectively gives witness data a 75 percent discount compared to traditional data. A practical effect on transaction fee incentives is that SegWit transactions are significantly cheaper to send than legacy transactions. This encourages users to adopt the more efficient format and helps limit the growth of the data that all full nodes must keep in memory to track spendable coins.

Part (d)

d) Describe how SegWit enables future extensibility of script versions without requiring a hard fork. Provide one example of how witness versioning allows new script types to be introduced in a backward-compatible way. (2)

SegWit enables future extensibility through witness versioning, where the first byte of a witness program defines how that script should be interpreted. This allows new script rules to be introduced through soft forks because older nodes do not validate the newer witness rules, while upgraded nodes enforce them. This makes the upgrade backward-compatible without requiring a hard fork. For example, Taproot was introduced as witness version 1 and added Schnorr signatures and MAST while remaining compatible with existing SegWit infrastructure.

Question 2

Bitcoin Scaling for Payments and Privacy Considerations (12)

Bitcoin's base layer has limited throughput but supports several techniques to scale payments and improve user privacy.

Part (a)

a) Describe why Bitcoin's base layer cannot scale to global, high-frequency payments. Include discussion of block size, block interval, and validation constraints. (2)

Bitcoin's base layer is restricted by a hard-coded block size limit and an average block interval of ten minutes, which results in a maximum throughput of roughly seven transactions per second. This capacity is insufficient for global, high-frequency payments because it creates a bottleneck where users must compete for limited space through high fees. Additionally, the validation constraint requires every full node to download and verify every transaction globally. If throughput were increased to match traditional payment processors, the hardware requirements for running a node would become so high that the network would likely centralize around a few powerful data centers.

Part (b)

b) Explain how the Lightning Network improves payment scalability. In your answer, describe:

- how off-chain payments reduce on-chain load,
- how multi-hop routed payments work, and
- the role of HTLCs in atomic forwarding. (3)

The Lightning Network improves scalability by moving payments off-chain, meaning that thousands of transactions can occur between two parties while only two transactions, the opening and closing of a channel, ever touch the blockchain. Multi-hop routed payments allow users to send funds to anyone else in the network by passing the payment through a chain of interconnected channels. This process is secured by Hashed Time-Locked Contracts (HTLCs), which use a secret preimage and a hashlock to ensure atomic forwarding. This ensures that either every node in the path successfully forwards the payment and receives their share, or the entire payment fails and the funds are returned.

Part (c)

c) Identify two privacy weaknesses of base-layer Bitcoin transactions related to: a. input/output linkability, b. address reuse. (2)

One primary privacy weakness is input and output linkability, where an observer can analyze the transaction graph to determine which addresses likely belong to the same entity based on how they are combined as inputs. A second weakness is address reuse, which occurs when a user receives multiple payments to the same public address. This makes it trivial for an observer to

aggregate the total balance and transaction history of that specific user, effectively stripping away the pseudonymity that Bitcoin is intended to provide.

Part (d)

d) Give two techniques—wallet-level or protocol-level—that improve privacy for everyday Bitcoin payments. Explain briefly how each helps. (2)

Wallet-level techniques like Hierarchical Deterministic (HD) wallets improve privacy by automatically generating a new address for every transaction, which prevents simple address reuse. At the protocol level, techniques like CoinJoin allow multiple users to combine their transactions into a single large transaction with many inputs and outputs. This breaks the link between specific inputs and outputs, making it difficult for chain analysis tools to determine which sender paid which recipient.

Part (e)

e) Compare the privacy characteristics of on-chain payments versus off-chain Lightning payments. In particular, who learns the payment details in each case, and what information is hidden from observers? (3)

On-chain payments are entirely public, meaning any observer with a block explorer can see the sender's address, the recipient's address, and the exact amount transferred. In contrast, Lightning payments are much more private. Only the nodes directly involved in the payment path learn any details about the transaction. Even then, intermediate nodes only know who sent them the funds and where to send them next, but they do not necessarily know the original sender or the final destination. Observers who are not part of the payment path see nothing at all.

Question 3

MAST (Merkelized Abstract Syntax Trees) and Script Commitments (10)

Taproot uses MAST to commit to multiple possible spending scripts while revealing only the branch that is actually executed. This greatly improves privacy and reduces on-chain data.

Part (a)

a) Explain the key idea behind MAST. In your answer, describe how Taproot commits to a set of possible scripts, how only the executed script branch is revealed, and why this provides better privacy than traditional P2WSH script commitments. (3)

The key idea behind Merkelized Abstract Syntax Trees (MAST) is to break a complex spending script into a set of individual conditions, each hashed and placed into a Merkle tree. In Taproot, the root of this Merkle tree is committed to the public key of the output. When a user spends the funds, they only reveal the specific script branch they are using and the Merkle proof connecting it to the root. This provides better privacy than P2WSH because unused branches, such as emergency recovery paths or complex multi-sig conditions, remain completely hidden from the blockchain.

Part (b)

b) Consider a Taproot output with three possible spending conditions represented by the following witness scripts:

$S1 = \langle pkA \rangle \text{ OP CHECKSIG},$
 $S2 = \text{OP IF } \langle pkB \rangle \text{ OP CHECKSIG OP ELSE } \langle pkC \rangle \text{ OP CHECKSIG OP ENDIF},$
 $S3 = \langle pkD \rangle \langle pkE \rangle 2 \text{ OP CHECKMULTISIG}.$

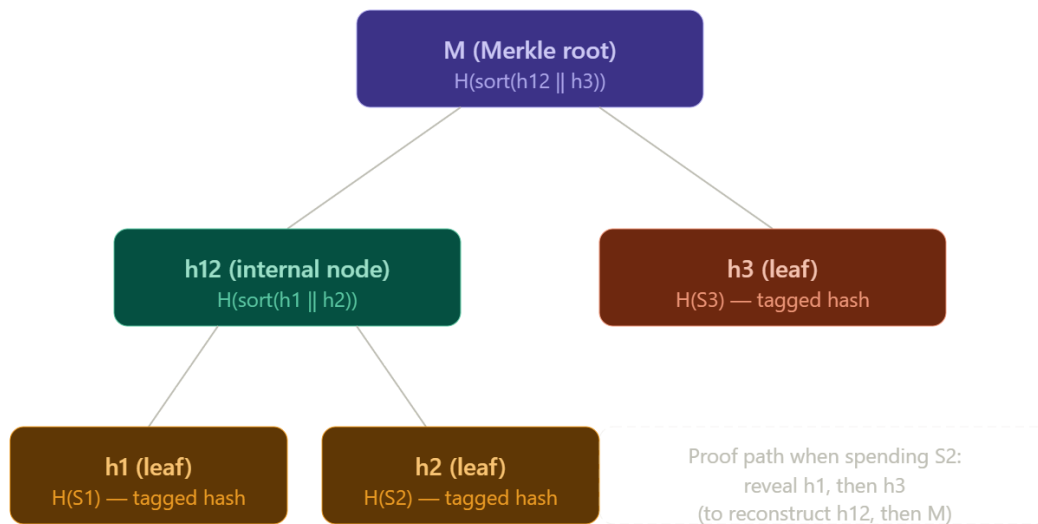
Construct the MAST (Merkle tree) over the script hashes of these three scripts. (2)

- Compute the leaf hashes $h1 = H(S1)$, $h2 = H(S2)$, $h3 = H(S3)$.
- Draw the Merkle tree assuming a standard taptree-style Merkle construction (no leaf duplication).
- Compute the internal node hashes and the final Merkle root M .

Let:

$h1 = H(S1)$
 $h2 = H(S2)$
 $h3 = H(S3)$

Using a taptree-style construction with no leaf duplication:



So the internal hashes are:

$$h12 = H(h1 \parallel h2)$$

$$M = H(h12 \parallel h3)$$

where M is the final Merkle root committed inside the Taproot output key.

Part (c)

c) Suppose the spender uses only script S2. Describe exactly what information must be included in the witness for a script-path Taproot spend, and show the Merkle proof path the spender must provide using the tree you constructed in part (b). (3)

If the spender uses script S2, the witness stack must contain the following items in order: The arguments to satisfy S2 come first. If spending via pkB: <sigB> then 1 (to select the OP_IF branch). If spending via pkC: <sigC> then 0 (to select the OP_ELSE branch). Next comes the full script S2 itself. Finally, the control block. The control block must explicitly contain three things: a version byte (0xC0 for Tapscript leaf version 0xC0), the internal public key P in its 32-byte x-only form, and the Merkle proof hashes in order. Since S2 is a leaf paired with S1 to form h12, and h12 is then paired with h3 to form M, the proof hashes are h1 followed by h3. The verifier then independently recomputes $h2 = H(S2)$, combines it with h1 to get $h12 = H(\text{sort}(h1 \parallel h2))$, and combines that with h3 to reconstruct $M = H(\text{sort}(h12 \parallel h3))$. If the reconstructed root matches the commitment inside the Taproot output key Q, the script path is proven valid. So the complete witness for the pkB case is:

<sigB>

<1>

<S2>

<control_block: 0xC0 || P || h1 || h3>

Part (d)

d) Explain why, even though three scripts exist, a chain observer learns nothing about S1 or S3 when S2 is used for the spend. (2)

A chain observer only sees the executed script S2, its arguments, and the Merkle proof hashes such as h1 and h3. They do not see the actual scripts S1 or S3. Since cryptographic hashes are one-way, the observer cannot recover whether S1 or S3 were simple signatures, multisig scripts, timelocks, or recovery conditions. At most, the observer learns that other committed branches existed, but not their contents.

Question 4

HTLC Scripts and Time-Locked Spending Under Taproot (12)

Hashed Time-Locked Contracts (HTLCs) allow conditional payments based on the revelation of a preimage or on a timeout refund. Taproot enables implementing HTLCs either through script-path spending (with a taptree leaf) or through cooperative key-path spending when both parties agree.

Part (a)

a) Explain the purpose of an HTLC in a multi-hop payment setting (e.g., Lightning). In your answer, describe the roles of the hashlock and timelock, and why the timelock must be relative or absolute depending on the protocol design. (2)

In a multi-hop setting, HTLCs ensure that a payment is atomic across the entire route. The hashlock forces the recipient to reveal a secret preimage to claim the funds, which then allows every preceding node in the path to claim their funds in turn. The timelock acts as a safety mechanism, allowing participants to reclaim their funds if the preimage is not revealed within a certain window. These timelocks must be absolute or relative depending on whether the contract is based on a specific block height or the time elapsed since the transaction was confirmed on-chain

Part (b)

b) Consider a Taproot output between Alice and Bob with internal key P and a taptree consisting of a single script leaf representing the HTLC. Let the payment hash be $h = H(x)$ and let the timeout be enforced by a timelock T . Construct a Taproot-compatible script-path HTLC leaf script that allows:

- Alice to spend the output if she learns the preimage x before time T , and
- Bob to refund the output after time T .

You may assume standard opcodes such as:

OP_CHECKSIG, OP_HASH160, OP_EQUAL, OP_CHECKLOCKTIMEVERIFY, and OP_DROP. (3)

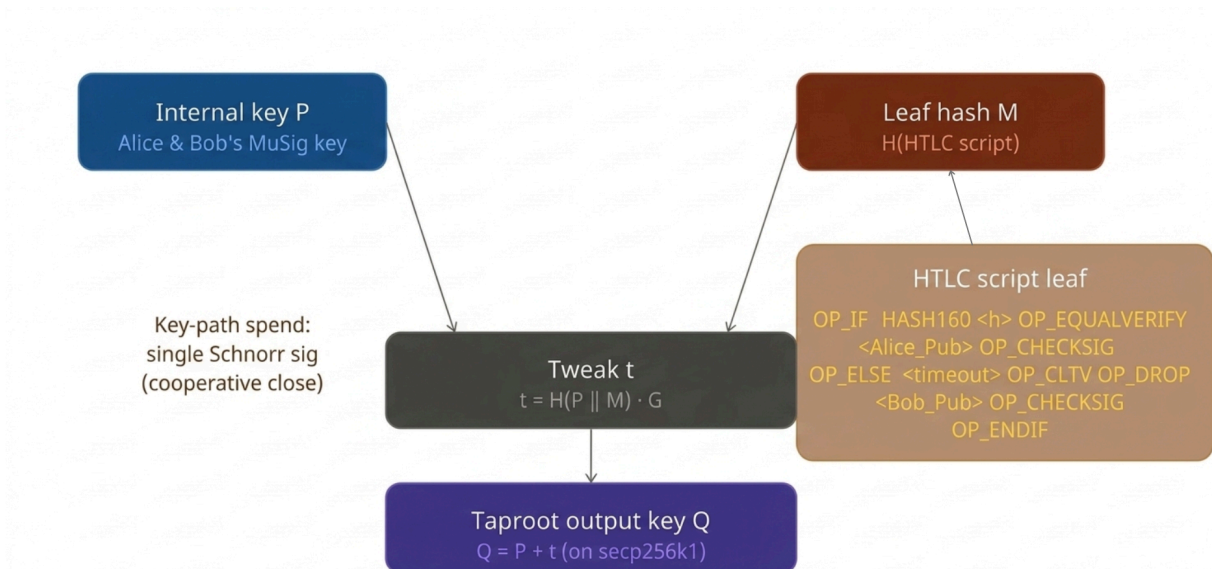
```
OP_IF
  OP_HASH160 <h> OP_EQUALVERIFY
  <Alice_Pub> OP_CHECKSIG
OP_ELSE
  <T> OP_CHECKLOCKTIMEVERIFY OP_DROP
  <Bob_Pub> OP_CHECKSIG
OP_ENDIF
```

Alice spends using the OP_IF branch by providing the preimage x such that $\text{HASH160}(x) = h$, along with her signature. Bob spends through the OP_ELSE branch after time T by providing his signature.

Part (c)

c) Draw the Taproot taptree for this output (one script leaf), label the leaf hash, and show how it combines with the internal key P to form the tweaked Taproot output key Q. (2)

The taptree for this output consists of a single leaf containing the hash of the HTLC script. To form the tweaked Taproot output key Q, the internal public key P is combined with the Merkle root M. The formula for this tweak is $Q = P + H(P \parallel M)G$, where G is the generator point of the elliptic curve. This construction allows the output to be spent either by the internal key or by the script path.



Part (d)

d) Suppose the HTLC output is spent cooperatively via the key-path with a single Schnorr signature. What information does a blockchain observer learn about:

- whether this was an HTLC at all,
- whether a timelock existed,
- whether a hash preimage was involved?

Explain why Taproot provides this level of privacy. (2)

Key-path Privacy If the output is spent cooperatively via the key-path, a blockchain observer learns nothing about the nature of the transaction. They see only a single Schnorr signature and a standard-looking public key. They cannot tell if there was an HTLC, a timelock, or a hashlock involved. Taproot provides this level of privacy by making complex smart contracts look identical to simple single-signature payments when the parties involved agree to the outcome

Part (e)

e) Suppose Alice spends via the script-path before time T using the preimage x. Write the full witness stack (in logical order) required for this Taproot script-path spend, including the Merkle proof/control block. (3)

To spend via the script-path before time T using preimage x, Alice must construct the witness stack so that items are placed in the order the script will consume them. The HTLC script begins with OP_IF, which pops the top stack item to decide the branch. Therefore the branch selector must sit above the preimage on the stack at evaluation time.

The correct witness stack in order from bottom to top is:

```
<Alice_sig>      ← satisfies <Alice_Pub> OP_CHECKSIG
<x>              ← satisfies OP_HASH160 <h> OP_EQUALVERIFY
<1>              ← consumed by OP_IF to select the hashlock branch
<HTLC_script>    ← the full serialised script being executed
<control_block>  ← version byte (0xC0) || internal key P || (no additional proof hashes, since
this is a single-leaf taptree and M = H(HTLC_script) directly)
```

The key ordering point is that <1> must be above <x>, because OP_IF pops the top item first. If the selector were below the preimage, OP_IF would consume x instead of 1 and the script would fail. The control block here is simpler than Q3c, because there is only one leaf in the taptree, there are no sibling hashes to include; the Merkle root IS the leaf hash, so the proof path is empty.

Question 5

Lightning Revocation Mechanisms and Punishment for Old-State Broadcasts (12)

Two-party payment channels require both parties to update their channel balances off-chain while preventing either participant from broadcasting an outdated commitment transaction. The Lightning Network uses a revocation-based punishment mechanism to enforce correct behavior.

Part (a)

a) Explain why outdated channel states (old commitment transactions) are dangerous. Describe what would happen if a party could safely publish an earlier state without penalty. (2)

Outdated channel states are dangerous because they represent balances that are no longer accurate. If a party could safely publish an earlier state without penalty, they would be incentivized to broadcast whichever state gave them the highest balance. For instance, if Alice had 1 BTC in an old state but only has 0.1 BTC in the current state, she would attempt to broadcast the old state to steal 0.9 BTC from Bob, effectively performing a double-spend of the off-chain funds.

Part (b)

b) In the simplified Lightning model used in class, each commitment transaction contains a revocation key whose private secret is revealed only when that state becomes old. Explain how a new state revokes the prior one. In your explanation, include:

- the role of the revocation secret for each state,
- how the counterparty uses it to gain control of the revocation key,
- and why revealing this secret makes an older state unsafe to publish. (3)

A new state revokes a prior one through the exchange of revocation secrets. Each commitment transaction includes an output that can be spent immediately by the counterparty if they possess a specific revocation secret. When Alice and Bob agree on a new state, Alice gives Bob the revocation secret for her previous state. This makes the older state unsafe to publish because Bob now has the cryptographic key necessary to take all the funds in that output if Alice ever broadcasts it to the network.

Part (c)

c) Suppose Alice and Bob open a channel. When they sign state $i + 1$, Alice reveals the revocation secret for state i , and Bob stores it. If Alice later broadcasts state i , which output can Bob take, and why? Describe how the timelock on that output enables Bob to punish Alice. (3)

If Alice broadcasts state i after it has been revoked, she is forced to wait for a relative timelock (CSV) before she can claim her funds. During this delay, Bob can use the revocation secret he stored to broadcast a justice transaction. This transaction spends Alice's output immediately because the revocation path of the script does not have a timelock. The delay on Alice's side

ensures that Bob has a window of opportunity to detect the cheating attempt and claim the entirety of the channel balance as a penalty.

Part (d)

d) Construct a simplified script (in Bitcoin Script) for a revocable output that pays either:

- the original owner after a timelock expires, or
- the counterparty immediately if they know the revocation secret. (2)

A simplified script for a revocable output would look like this:

`OP_IF <Revocation_Pub> OP_CHECKSIG OP_ELSE <Delay_T>`

`OP_CHECKSEQUENCEVERIFY OP_DROP <Owner_Pub> OP_CHECKSIG OP_ENDIF.` In this script, the counterparty can spend the funds immediately if they have the signature for the Revocation Public Key, which is derived from the revocation secret. Otherwise, the original owner must wait for the duration of the delay T before their signature becomes valid.

Part (e)

e) Describe what Bob includes in his punishment (justice) transaction when Alice broadcasts an old state, and explain why this mechanism creates strong economic incentives to only broadcast the latest state. (2)

When Alice broadcasts old state i , Bob constructs a justice transaction that spends Alice's time-locked output from that commitment transaction before the CSV delay expires. In the witness for that input, Bob provides his signature using the revocation key and the value `1` to select the revocation branch of the script, which has no timelock and is therefore immediately spendable. The revocation key itself is derived from the revocation secret that Alice revealed when state $i+1$ was negotiated. Bob uses that secret to reconstruct the private key corresponding to the `Revocation_Pub` in Alice's script.

Critically, Bob's justice transaction does not merely claim Alice's direct output. If the old commitment also had any HTLC outputs in-flight at the time of that state, Bob can sweep those as well, since each HTLC output in a revoked state also has a revocation path that becomes accessible to Bob once he holds the revocation secret. The justice transaction therefore collects the entirety of the channel capacity which is Alice's balance, Bob's own balance (already his), and any in-flight amounts sending everything to Bob's wallet.

This creates an extremely strong economic disincentive. A cheating party does not merely fail to gain from broadcasting an old state as they risk losing every satoshi they have locked in the channel. Because the potential loss always equals the full channel capacity, which is by definition larger than any gain from cheating, the rational strategy is always to broadcast only the latest valid state.

Question 6

Lightning Network Fundamentals: Payment Channels and Commitment Transactions (14)

Two parties, Alice and Bob, set up a bidirectional payment channel using a funding transaction and off-chain commitment transactions representing their evolving balances.

Part (a)

a) Describe the structure and purpose of the funding transaction in a Lightning channel. Why must it be a 2-of-2 multisig output, and how does this fact enable trust-minimized updates? (3)

The funding transaction is a standard Bitcoin transaction that creates a 2-of-2 multisig output on the blockchain. Its purpose is to lock a specific amount of Bitcoin into the channel, which acts as the total capacity for all future off-chain transfers. It must be a 2-of-2 multisig to ensure trust-minimized updates, as neither Alice nor Bob can move the funds without the explicit consent and signature of the other party.

Part (b)

b) Define what a commitment transaction is. Explain:

- who signs it,
- when it is valid to broadcast,
- why each party holds a slightly different commitment transaction. (3)

A commitment transaction is an off-chain transaction that spends the funding transaction and pays the current channel balance back to Alice and Bob. Each party holds a version that they can broadcast if they need to close the channel unilaterally. It is signed by the counterparty in advance, so the holder already has enough authorization to publish it.

Each party holds a slightly different commitment transaction because the broadcaster's own output is delayed, while the counterparty's output is immediately spendable. This difference helps assign blame and gives the counterparty time to punish old-state broadcasts.

Part (c)

c) Explain how off-chain payments change the channel balance without broadcasting transactions. Describe the update process that ensures both parties end up with the latest valid state and that older states cannot be safely broadcast. (3)

Off-chain payments change the channel balance by creating a new pair of commitment transactions and then invalidating the previous ones. The update process involves both parties signing the new state and then exchanging the revocation secrets for the old state. This ensures that both parties always possess the latest valid state and that any attempt to broadcast an older, revoked state can be met with a punishment transaction.

Part (d)

d) Consider a channel where Alice initially contributes 1 BTC and Bob contributes 0 BTC. They then perform the following off-chain transfers:

- a. Alice pays Bob 0.3 BTC,
- b. Bob pays Alice 0.1 BTC,
- c. Alice pays Bob 0.25 BTC.

Compute the final off-chain channel balance for each party and explain how this is reflected in the final pair of commitment transactions. (3)

The initial state is Alice having 1.0 BTC and Bob having 0.0 BTC. After Alice pays Bob 0.3 BTC, the balances are 0.7 and 0.3. After Bob pays Alice 0.1 BTC, the balances become 0.8 and 0.2. Finally, after Alice pays Bob 0.25 BTC, the final off-chain balances are 0.55 BTC for Alice and 0.45 BTC for Bob. This is reflected in the final pair of commitment transactions, where each transaction allocates those specific amounts to the respective parties.

Part (e)

e) Give one reason why commitment transactions must include timelocks, and describe what could go wrong if timelocks were not present. (2)

Commitment transactions must include timelocks to provide a window for the revocation mechanism to function. If timelocks were not present, a party could broadcast an old state and spend their output immediately. Without a delay, the victim would have no time to broadcast a justice transaction, making the punishment mechanism useless and rendering the entire Lightning Network insecure against dishonest participants.